In [33]:
```
import random
import re
```

In [35]:
```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score, classification_report
```

In [37]:
```
training_data = {
    "greeting": [
        "hello", "hi", "hey", "good morning",
        "good evening", "how are you", "hi there"
    ],
    "goodbye": [
        "bye", "goodbye", "see you later",
        "talk to you later", "exit", "quit"
    ],
    "thanks": [
        "thanks", "thank you", "thanks a lot",
        "that's helpful", "great help"
    ],
    "bot_identity": [
        "who are you", "what are you",
        "what is your name", "are you a bot"
    ],
    "help": [
        "help me", "what can you do",
        "show your features", "how can you help"
    ],
    "python": [
        "what is python", "explain python",
        "why use python", "python programming"
    ],
    "machine_learning": [
        "what is machine learning",
        "explain machine learning",
        "how does machine learning work",
        "what is ML"
    ],
    "nlp": [
        "what is NLP",
        "explain natural language processing",
        "what does NLP mean",
        "tell me about text processing"
    ],
    "project_ideas": [
        "suggest a Python project",
        "give me project ideas",
        "machine learning project ideas",
        "NLP project ideas"
    ],
    "career": [
        "how do I get a data science job",
        "data analyst career advice",
        "how to prepare for ML interviews",
        "help with my resume"
    ],
```

```
    "hours": [
        "how long will this take",
        "how much time do I need",
        "how many hours should I study"
    ]
}
```

In [39]:
```
responses = {
    "greeting": [
        "Hello! How can I help you?",
        "Hi there! Ask me about Python, ML, or NLP.",
        "Hey! What would you like to learn today?"
    ],
    "goodbye": [
        "Goodbye! Keep practising Python.",
        "See you later. Good luck!"
    ],
    "thanks": [
        "You're welcome!",
        "Happy to help!",
        "Glad that was useful."
    ],
    "bot_identity": [
        "I am a Python chatbot using machine learning to classify user in
    ],
    "help": [
        "I can answer questions about Python, ML, NLP, projects and caree
        "Commands: /help, /history, /evaluate, /quit."
    ],
    "python": [
        "Python is a general-purpose programming language used in automat
        "data analysis, web development and machine learning."
    ],
    "machine_learning": [
        "Machine learning allows computers to learn patterns from data "
        "and use them to make predictions."
    ],
    "nlp": [
        "NLP stands for Natural Language Processing. It helps computers "
        "understand and process human language."
    ],
    "project_ideas": [
        "Try building a spam detector, sentiment analyser, FAQ chatbot "
        "or movie recommendation system."
    ],
    "career": [
        "Build practical projects, practise Python and SQL, and prepare "
        "to explain your projects clearly in interviews."
    ],
    "hours": [
        "Try studying for 1-2 focused hours daily and adjust according "
        "to your schedule."
    ]
}
```

In [41]:
```
def normalize_text(text):
    text = text.lower()
    text = re.sub(r"\s+", " ", text)
    return text.strip()
```

In [43]:
```
texts = []
labels = []

for intent, examples in training_data.items():
    for example in examples:
        texts.append(normalize_text(example))
        labels.append(intent)
```

In [45]:
```
X_train, X_test, y_train, y_test = train_test_split(
    texts,
    labels,
    test_size=0.25,
    random_state=42,
    stratify=labels
)
```

In [47]:
```
# 6. Build ML pipeline
model = Pipeline([
    ("tfidf", TfidfVectorizer(
        ngram_range=(1, 2),
        sublinear_tf=True
    )),
    ("classifier", LogisticRegression(max_iter=1000))
])
```

In [49]:
```
# 7. Train and evaluate model
model.fit(X_train, y_train)

predictions = model.predict(X_test)

print("MODEL EVALUATION")
print("Accuracy:", round(accuracy_score(y_test, predictions), 3))

print(classification_report(
    y_test,
    predictions,
    zero_division=0
))
```

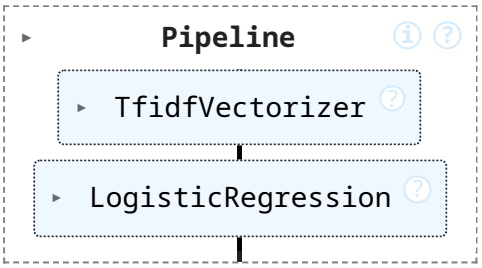
```
MODEL EVALUATION
Accuracy: 0.615
                  precision    recall  f1-score   support

    bot_identity       0.50      1.00      0.67         1
          career       0.00      0.00      0.00         1
         goodbye       1.00      0.50      0.67         2
        greeting       0.20      0.50      0.29         2
            help       1.00      1.00      1.00         1
           hours       0.00      0.00      0.00         1
machine_learning       1.00      1.00      1.00         1
             nlp       0.00      0.00      0.00         1
   project_ideas       1.00      1.00      1.00         1
          python       1.00      1.00      1.00         1
          thanks       1.00      1.00      1.00         1

        accuracy                           0.62        13
       macro avg       0.61      0.64      0.60        13
    weighted avg       0.61      0.62      0.58        13
```

In [51]:
```python
# 8. Train final model on complete dataset
model.fit(texts, labels)
```

Out[51]:



In [53]:
```python
# 9. Conversation history
conversation_history = []
```

In [55]:
```python
# 10. Generate chatbot response
def get_reply(user_text):

    cleaned_text = normalize_text(user_text)

    if not cleaned_text:
        return "Please type something."

    predicted_intent = model.predict([cleaned_text])[0]

    probabilities = model.predict_proba([cleaned_text])[0]

    confidence = max(probabilities)

    if confidence < 0.15:
        return (
            "I'm not sure I understood. "
            "Try asking about Python, ML, NLP or careers."
        )

    return random.choice(responses[predicted_intent])
```

In [57]:
```python
# 11. Help command
def show_help():

    print("\nAvailable commands:")
    print("/help      - Show commands")
    print("/history   - Show conversation history")
    print("/evaluate  - Evaluate model")
    print("/quit      - Exit chatbot")

    print("\nTopics: Python, ML, NLP, projects and careers")
```

In [59]:
```python
# 12. History command
def show_history():

    if not conversation_history:
        print("No conversation history yet.")
        return

    for user_message, bot_reply in conversation_history[-10:]:
        print("You:", user_message)
        print("Bot:", bot_reply)
```

In [61]:
```python
# 13. Evaluation command
def evaluate_model():

    predictions = model.predict(X_test)

    print(
        "Held-out accuracy:",
        round(accuracy_score(y_test, predictions), 3)
    )

    print("Note: Small dataset, so this score is only indicative.")
```

In [63]:
```python
# 14. Main chatbot
def main():

    print("\n===== INTERMEDIATE NLP CHATBOT =====")
    print("Type /help for commands.")
    print("Type /quit to exit.")

    while True:

        user_message = input("\nYou: ").strip()

        if user_message.lower() in ["/quit", "quit", "exit"]:
            print("Bot: Goodbye!")
            break

        elif user_message.lower() == "/help":
            show_help()
            continue

        elif user_message.lower() == "/history":
            show_history()
            continue

        elif user_message.lower() == "/evaluate":
```

```
            evaluate_model()
            continue

        reply = get_reply(user_message)

        conversation_history.append(
            (user_message, reply)
        )

        print("Bot:", reply)


if __name__ == "__main__":
    main()
```

```
===== INTERMEDIATE NLP CHATBOT =====
Type /help for commands.
Type /quit to exit.
Bot: Hey! What would you like to learn today?
Bot: Python is a general-purpose programming language used in automation,
data analysis, web development and machine learning.
Bot: Glad that was useful.
Bot: Goodbye!
```

In [ ]: